

Boas Práticas de Programação

Prof. Dr. Debora B. Aires

SOLID

- SOLID: é um acrônimo que representa cinco princípios de design de software orientado a objetos que visam criar sistemas mais flexíveis, extensíveis e fáceis de manter. Cada letra corresponde a um princípio específico:
 - S (Single Responsibility Principle - Princípio da Responsabilidade Única): Cada classe deve ter uma única razão para mudar, ou seja, deve ter uma única responsabilidade.
 - O (Open/Closed Principle - Princípio Aberto/Fechado): As entidades de software (classes, módulos, etc.) devem ser abertas para extensão, mas fechadas para modificação. Isso promove a adição de novas funcionalidades sem alterar o código existente.
 - L (Liskov Substitution Principle - Princípio da Substituição de Liskov): As instâncias de uma classe derivada devem ser substituíveis pelas instâncias da classe base sem afetar a integridade do sistema.
 - I (Interface Segregation Principle - Princípio da Segregação de Interfaces): Clientes não devem ser forçados a depender de interfaces que eles não usam. Em outras palavras, as interfaces devem ser específicas para as necessidades dos clientes.
 - D (Dependency Inversion Principle - Princípio da Inversão de Dependência): Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações. Além disso, abstrações não devem depender de detalhes, mas detalhes devem depender de abstrações.
- RESUMO: SOLID promove a modularidade e facilita a manutenção do código. Ajuda a evitar acoplamento excessivo e torna os sistemas mais flexíveis para mudanças.

SOLID

- Veja esse vídeo:
 - https://www.youtube.com/watch?v=6SfrO3D4dHM&ab_channel=FilipeDeschamps
- Escute esse podcast:
 - <https://www.hipsters.tech/solid-codigo-bom-e-bonito-hipsters-ponto-tech-219/>

Boas práticas: SOLID

Fonte da imagem:

<https://blog.betrybe.com/linguagem-de-programacao/solid-cinco-principios-poo/>

S

SRP - Single Responsibility Principle
(Princípio da **responsabilidade única**)

O

OCP - Open-Closed Principle
(Princípio **Aberto-Fechado**)

L

LSP - Liskov Substitution Principle
(Princípio da **substituição de Liskov**)

I

ISP - Interface Segregation Principle
(Princípio da **Segregação da Interface**)

D

DIP - Dependency Inversion Principle
(Princípio da **inversão da dependência**)

Benefícios da SOLID

- **Segurança:**
 - As classes e aplicação se tornam mais seguras graças a divisão de responsabilidades. Com o SOLID, não corremos o risco de ter classes “quebradas” por que alguém alterou algum método de outra classe.
- **Manutenção:**
 - Com responsabilidades divididas, fica muito mais fácil dar manutenção na aplicação
- **Reutilizável:**
 - Não é necessária ficar reescrevendo o mesmo código para executar uma determinada tarefa, basta estender de outra classe que já tenha a função que você precisa.

• Fonte da imagem:
<https://blog.betrybe.com/linguagem-de-programacao/solid-cinco-principios-poo/>

Princípio da responsabilidade única (coesão)

- **Definição:** Uma classe deve ter um, e apenas um, motivo para existir ou mudar, focando em uma única responsabilidade.
- **Exemplo:** Uma classe Calculadora que apenas realiza operações matemáticas, sem se responsabilizar por entrada/saída de dados.



Princípio da responsabilidade única

Refatorando..

```
class Order {  
  
    public function calculateTotalSum() { /*...*/ }  
    public function getItems() { /*...*/ }  
    public function getItemCount() { /*...*/ }  
    public function addItem($item) { /*...*/ }  
    public function deleteItem($item) { /*...*/ }  
  
    public function printOrder() { /*...*/ }  
    public function showOrder() { /*...*/ }  
  
    public function load() { /*...*/ }  
    public function save() { /*...*/ }  
    public function update() { /*...*/ }  
    public function delete() { /*...*/ }  
}
```

```
class Order {  
    public function calculateTotalSum() { /*...*/ }  
    public function getItems() { /*...*/ }  
    public function getItemCount() { /*...*/ }  
    public function addItem($item) { /*...*/ }  
    public function deleteItem($item) { /*...*/ }  
}  
  
class OrderRepository {  
    public function load($orderId) { /*...*/ }  
    public function save($order) { /*...*/ }  
    public function update($order) { /*...*/ }  
    public function delete($order) { /*...*/ }  
}  
  
class OrderViewer {  
    public function printOrder($order) { /*...*/ }  
    public function showOrder($order) { /*...*/ }  
}
```

+

•

○

Exercício prático: Aplicando o Princípio da Responsabilidade Única (SRP) do SOLID

- util.DialogUtil já cumprem essa responsabilidade;
- Porém ainda é possível aplicar o Princípio da Responsabilidade Única (SRP) do SOLID no nosso Controller.
- Atualmente, essa classe tem múltiplas responsabilidades, o que pode torná-la mais difícil de manter, testar e estender.
- A linha entre o que é "essencial" para um controller e o que deve ser delegado a um serviço nem sempre é óbvia e pode gerar discussões.
- Identificando as Responsabilidades Atuais:
 - Controle da Interface do Usuário: Manipulação dos elementos visuais (text fields, table view, buttons), configuração de placeholders e listeners.
 - Validação de Dados: Lógica para validar os campos de entrada (campos vazios, formato de e-mail).
 - Acesso a Dados: Criação e uso de UsuarioDAO para persistir e recuperar dados de usuários.
 - Manipulação de Dados: Criação de objetos UsuarioDTO, população da tabela e preenchimento dos campos.
 - Feedback ao Usuário: Exibição de mensagens de aviso, informação e confirmação.

Para aplicar o SRP, podemos refatorar a classe Controller dividindo suas responsabilidades em classes separadas.

- Classe de Validação (UsuarioValidator):
 - Responsabilidade: Conter a lógica de validação dos dados do usuário (campos obrigatórios, formato de e-mail, etc.).
 - O método validarCampos() e validarEmail() poderiam ser movidos para essa classe.
- Classe de Serviço (UsuarioService):
 - Responsabilidade: Conter a lógica de negócios relacionada aos usuários. Isso incluiria a criação de objetos UsuarioDTO, a chamada ao UsuarioDAO para persistência e recuperação, e potencialmente outras regras de negócio.
 - Parte dos métodos btnCadastrarAction, btnAtualizarAction, btnExcluirAction (a parte de manipulação de dados e interação com o DAO) e listarUsuarios() poderiam ser delegados a outras classe.

Antes (exemplo de trecho de código não refatorado):

```
private boolean validarCampos() {  
    if (txtNome.getText().isEmpty() || txtEmail.getText().isEmpty() ||  
        txtSenha.getText().isEmpty() || txtLogin.getText().isEmpty()) {  
        showWarning("Preencha todos os campos antes de prosseguir.");  
        return false;  
    }  
    if (!validarEmail(txtEmail.getText())) {  
        showWarning("Digite um e-mail válido (exemplo@dominio.com)!");  
        return false;  
    }  
    return true;  
}  
  
private boolean validarEmail(String email) {  
    return Pattern.matches("^([\\w.-]+@[\\w.-]+\\.\\w+$", email);  
}
```

Veja exemplos de como ficaria o código refatorado:

```
...
import validator.UsuarioValidator; // Importa o seu novo validador
...
public class FXMLDocumentController implements Initializable {

    // Instancia o serviço e o validador
    private final UsuarioService usuarioService = new UsuarioService();
    private final UsuarioValidator usuarioValidator = new UsuarioValidator();

    @FXML
    private void btnCadastrarAction(ActionEvent event) {
        // Delega toda a validação para o UsuarioValidator
        if (!usuarioValidator.validarUsuario(txtNome.getText(), txtEmail.getText(), txtSenha.getText(), txtLogin.getText())) {
            return; // Se a validação falhar, para por aqui
        }

        // Se a validação passou, continua com a lógica de negócio
        UsuarioDTO objUsuario = new UsuarioDTO();
        objUsuario.setNome(txtNome.getText());
        objUsuario.setEmail(txtEmail.getText());
        objUsuario.setSenha(txtSenha.getText());
        objUsuario.setLogin(txtLogin.getText());

        usuarioService.cadastrarUsuario(objUsuario); // Chama o serviço
        limparCampos();
        listarUsuarios();
        showInformation("Usuário cadastrado com sucesso!");
    }
}
```

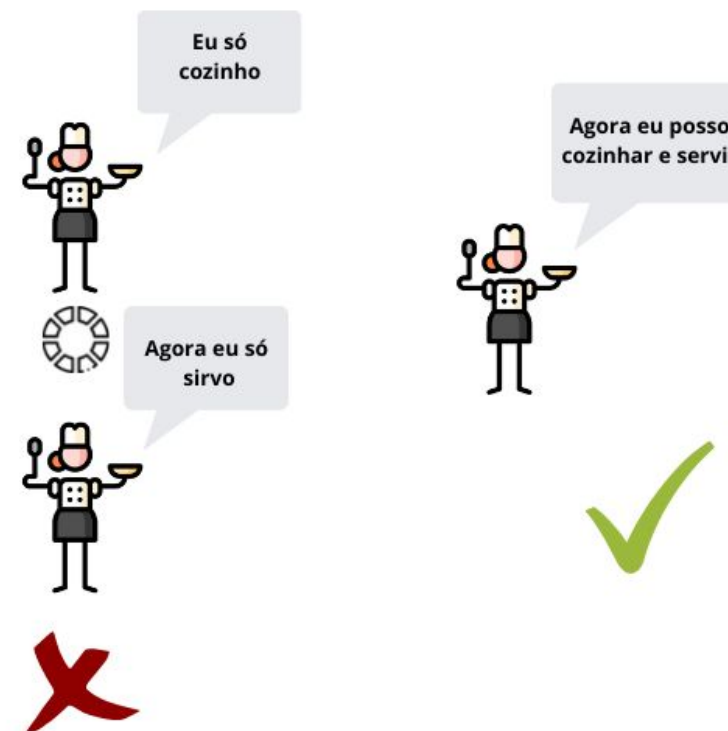
Aqui fica a continuação do seu código, não necessariamente esse, este é um exemplo!!!

Benefícios da Aplicação do SRP

- Maior Coesão: Cada classe tem um único motivo para mudar.
- Menor Acoplamento: As classes se tornam mais independentes, facilitando a manutenção e evitando efeitos colaterais em outras partes do sistema.
- Melhor Testabilidade: Classes menores e com responsabilidades bem definidas são mais fáceis de testar unitariamente.
- Maior Reusabilidade: As classes de serviço e validação podem ser reutilizadas em outras partes da aplicação, se necessário.
- Melhor Organização e Compreensão: O código se torna mais organizado e fácil de entender.

Princípio Aberto-Fechado (garante sistema extensível)

- **Definição:** Entidades de software (classes, módulos, funções) devem estar abertas para extensão, mas fechadas para modificação.
- **Exemplo:** Um sistema de relatórios onde você pode adicionar novos tipos de relatórios sem alterar o código existente, apenas estendendo a classe base de relatório.



Princípio Aberto-Fechado

```
class ContratoClt {
    public function salario() { /*...*/ }
}

class Estagio {
    public function bolsaAuxilio() { /*...*/ }
}

class FolhaDePagamento {
    protected $saldo;

    public function calcular( $contrato ) {
        if ( $contrato instanceof ContratoClt ) {
            $this->saldo = $contrato->salario();
        } else if ( $contrato instanceof Estagio ) {
            $this->saldo = $contrato->bolsaAuxilio();
        }
    }
}
```

Refatorando..

.

```
interface Remuneravel {
    public function remuneracao();
}

class ContratoClt implements Remuneravel {
    public function remuneracao() { /*...*/ }
}

class ContratoPj implements Remuneravel {
    public function remuneracao() { /*...*/ }
}

class Estagio implements Remuneravel {
    public function remuneracao() { /*...*/ }
}

class FolhaDePagamento {
    protected $saldo;

    public function calcular( Remuneravel $contrato ) {
        $this->saldo = $contrato->remuneracao();
    }
}
```

Que vantagem temos ao depender de interfaces e não de implementações?

Se um método muda a forma como realiza sua tarefa, desde que a interface se mantenha, não vamos precisar nos preocupar nem em editar o nosso código.

Exercício prático: Aplicando o Princípio Aberto/Fechado (OCP) do SOLID

- Analisando o nosso Controller, podemos identificar alguns pontos onde o OCP poderia ser aplicado:
 - Validação de Dados: Atualmente, a lógica de validação está diretamente no método `validarCampos()`. Se você precisar adicionar novas regras de validação (por exemplo, validar o formato do login, verificar se o login já existe), você precisará modificar esse método. Para aplicar o OCP, você poderia usar um sistema de validação mais flexível.
- Possível Solução: Criar uma interface `Validador<T>` e implementações concretas para cada tipo de validação (`NomeValidador`, `EmailValidador`, `SenhaValidador`, `LoginValidador`). O `validarCampos()` poderia iterar sobre uma lista de validadores, sem precisar ser modificado quando novos validadores forem adicionados.

Benefícios da Aplicação do OCP

- A ideia central é permitir a adição de novas funcionalidades sem a necessidade de alterar o código já existente e funcionando, o que ajuda a evitar a introdução de bugs e simplifica a manutenção

Veja
exemplos de
como ficaria
o código
refatorado:

```
package validator; // Certifique-se de que todas as classes estejam no mesmo pacote 'validator'

// Interface para definir um contrato de validação
interface Validador<T> {
    boolean validar(T valor); // O método validar recebe o valor para que o Validador possa ser mais genérico
    String getMensagemErro();
    T getValor(); // Novo método para retornar o valor que está sendo validado
}

class CampoObrigatorioValidador implements Validador<String> {
    private final String nomeCampo;
    private final String valor; // Armazena o valor a ser validado

    public CampoObrigatorioValidador(String nomeCampo, String valor) {
        this.nomeCampo = nomeCampo;
        this.valor = valor;
    }

    @Override
    public boolean validar(String valorAtual) { // O valor do parâmetro é o que será validado neste ciclo
        return this.valor != null && !this.valor.trim().isEmpty();
    }

    @Override
    public String getMensagemErro() {
        return "O campo " + nomeCampo + " deve ser preenchido.";
    }

    @Override
    public String getValor() {
        return valor;
    }
}

class EmailValidador implements Validador<String> {
    private static final String EMAIL_REGEX = "^[\\w.-]+@[\\w.-]+\\.\\w+$";
    private final Pattern pattern = Pattern.compile(EMAIL_REGEX);
    private final String email; // Armazena o e-mail a ser validado

    public EmailValidador(String email) {
        this.email = email;
    }

    @Override
    public boolean validar(String valorAtual) { // O valor do parâmetro é o que será validado neste ciclo
        return this.email != null && pattern.matcher(this.email).matches();
    }

    @Override
    public String getMensagemErro() {
        return "Digite um e-mail válido (exemplo@dominio.com)!";
    }

    @Override
    public String getValor() {
        return email;
    }
}
```

```
public class UsuarioValidator {  
  
    // Método principal de validação que combina todas as regras  
    public boolean validarUsuario(String nome, String email, String senha, String login) {  
        // Lista de validadores que serão aplicados sequencialmente  
        List<Validador<String>> validadores = new ArrayList<>();  
  
        // Adicionando os validadores de campos obrigatórios  
        validadores.add(new CampoObrigatorioValidador("Nome", nome));  
        validadores.add(new CampoObrigatorioValidador("E-mail", email));  
        validadores.add(new CampoObrigatorioValidador("Senha", senha));  
        validadores.add(new CampoObrigatorioValidador("Login", login));  
  
        // Adicionando o validador específico de formato de e-mail (aplicado ao campo email)  
        validadores.add(new EmailValidador(email));  
  
        // Itera sobre a lista de validadores  
        for (Validador<String> validador : validadores) {  
            // Cada validador testa seu valor específico  
            if (!validador.validar(validador.getValor())) { // O validador agora "conhece" o valor que vai validar  
                showWarning(validador.getMensagemErro()); // você usaria DialogUtil.showWarning(mensagem);  
                return false; // Retorna falso na primeira falha de validação  
            }  
        }  
        return true; // Todos os validadores passaram  
    }  
}
```

Princípio da substituição de Liskov

- **Definição:** As classes derivadas devem ser substituíveis pelas suas classes bases
- **Exemplo:** Aluno e professor herdam os atributos de uma classe pai Pessoa.



Princípio da substituição de Liskov

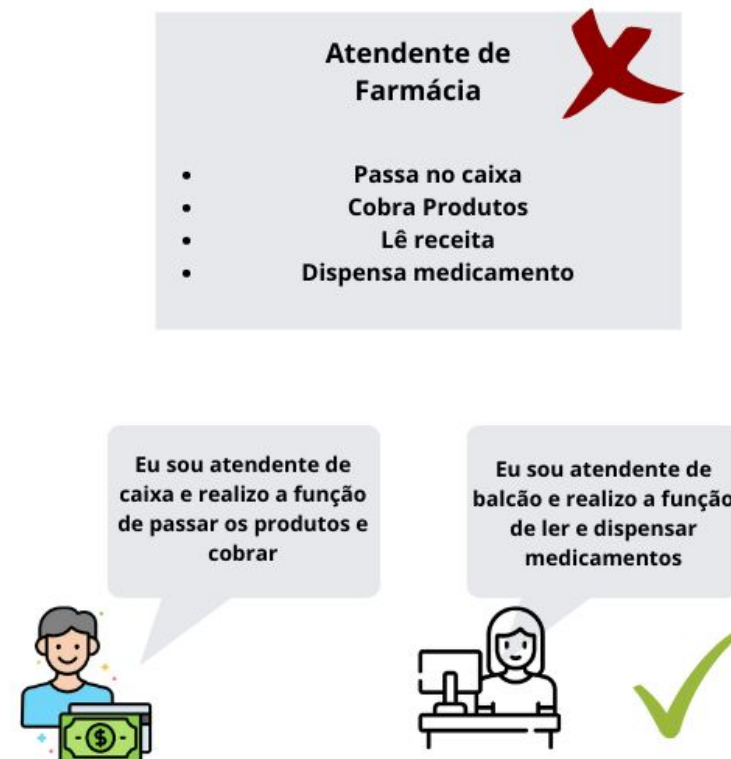
```
class A {  
    public function getNome() {  
        echo 'Meu nome é A';  
    }  
}  
  
class B extends A {  
    public function getNome() {  
        echo 'Meu nome é B';  
    }  
}  
  
$objeto1 = new A;  
$objeto2 = new B;  
  
function imprimeNome( A $objeto ) {  
    return $objeto->getNome();  
}  
  
imprimeNome($objeto1); // Meu nome é A  
imprimeNome($objeto2); // Meu nome é B
```


Princípio da Substituição de Liskov (LSP)

- o Princípio da Substituição de Liskov (LSP) **já está sendo aplicado** no nosso código, especialmente na maneira de validar com a interface Validador<T>.
- As classes filhas não devem quebrar o "contrato" definido pela classe pai (ou interface). Isso significa que elas não devem lançar exceções inesperadas, retornar valores de tipos diferentes do esperado, ou ter efeitos colaterais que não seriam esperados se a classe pai estivesse sendo usada.

Princípio da Segregação da Interface

- **Definição:** Classes não devem ser forçadas a depender de métodos que não usam.
- **Exemplo:** A classe ave não pode implementar voar() visto que nem todas as aves voam.



Princípio da Segregação da Interface

Refatorando..

```
interface Aves {
    public function setLocalizacao( $longitude, $latitude );
    public function setAltitude( $altitude );
    public function renderizar();
}

class Papagaio implements Aves {
    public function setLocalizacao( $longitude, $latitude ) { /*...*/ }

    public function setAltitude( $altitude ) { /*...*/ }

    public function renderizar() { /*...*/ }
}

class Pinguim implements Aves {
    public function setLocalizacao( $longitude, $latitude ) { /*...*/ }

    public function setAltitude( $altitude ) {
        // Não faz nada... Pinguins não voam!
    }

    public function renderizar() { /*...*/ }
}
```

```
interface Aves {
    public function setLocalizacao( $longitude, $latitude );
    public function renderizar();
}

interface AvesQueVoam extends Aves {
    public function setAltitude($altitude);
}

class Papagaio implements AvesQueVoam {
    public function setLocalizacao( $longitude, $latitude ) { /*...*/ }
    public function setAltitude( $altitude ) { /*...*/ }
    public function renderizar() { /*...*/ }
}

class Pinguim implements Aves {
    public function setLocalizacao( $longitude, $latitude ) { /*...*/ }
    public function renderizar() { /*...*/ }
}
```

Uma classe não deve ser obrigada a implementar um método de determinada interface, se ele não será útil. Para isso, uma interface deverá ser extraída apenas com os métodos necessários.

Princípio da Segregação da Interface

Refatorando..

```
interface Trabalhador {  
    void trabalhar();  
    void comer();  
    void dormir();  
    void gerenciarFuncionarios(); // Apenas gerentes fazem isso  
    void codificar();             // Apenas desenvolvedores fazem isso  
    void atenderTelefone();       // Apenas atendentes fazem isso  
}
```

```
interface TrabalhadorGeral {  
    void trabalhar();  
    void comer();  
    void dormir();  
}  
  
interface Gerente extends TrabalhadorGeral {  
    void gerenciarFuncionarios();  
}  
  
interface Desenvolvedor extends TrabalhadorGeral {  
    void codificar();  
}  
  
interface Atendente extends TrabalhadorGeral {  
    void atenderTelefone();  
}
```

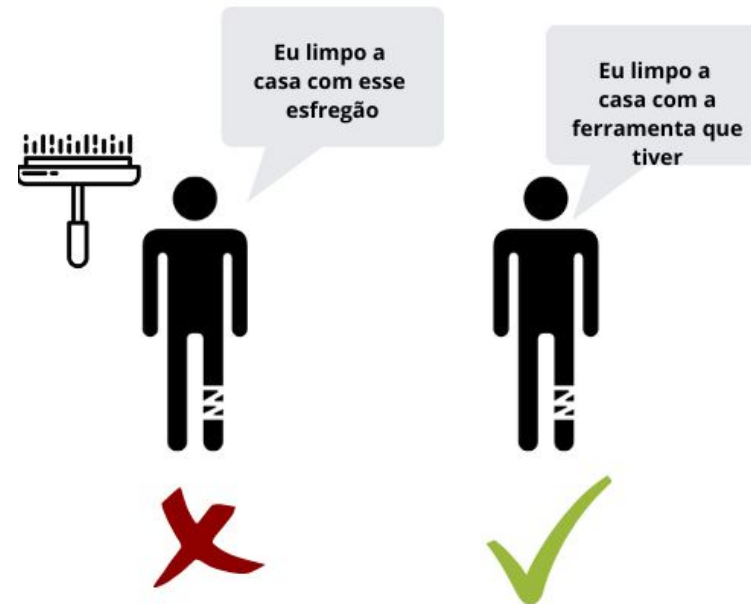
Uma classe não deve ser obrigada a implementar um método de determinada interface, se ele não será útil. Para isso, uma interface deverá ser extraída apenas com os métodos necessários.

Princípio da Segregação da Interface

- Se uma classe não precisa implementar um método de uma interface, essa classe não deveria ter que depender dessa interface. Isso evita que classes sejam "poluídas" com métodos que não lhes dizem respeito, tornando-as mais coesas e fáceis de manter.
- No nosso caso, com a interface `Validador<T>` e as classes `CampoObrigatorioValidador` e `EmailValidador`, a interface já é relativamente pequena e coesa.

Princípio da inversão da dependência

- **Definição:** Abstrações não devem depender de detalhes. Os detalhes devem depender das abstrações.
- **Exemplo:** Um sistema de notificação chama a função `INotificador()` ao invés de qual notificação específica é, por exemplo `EmailNotificador`, `SmsNotificador`.



O Princípio da Inversão da Dependência (DIP)

- O DIP afirma que:
 - Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.
 - Abstrações não devem depender de detalhes. Detalhes devem depender de abstrações.
- Em termos mais simples:
 - Classes que contêm lógica de negócios importante (módulos de alto nível) não devem depender diretamente de classes que lidam com detalhes de implementação (módulos de baixo nível, como acesso a banco de dados ou detalhes de UI).
 - Em vez disso, ambos devem depender de interfaces ou classes abstratas (abstrações). As implementações concretas (detalhes) devem ser fornecidas em tempo de execução.
- O objetivo principal do DIP é reduzir o acoplamento entre os módulos, tornando o sistema mais flexível e fácil de modificar.

Princípio da inversão da dependência

Refatorando..

```
use MySqlConnection;

class PasswordReminder {
    private $dbConnection;

    public function __construct() {
        $this->dbConnection = new MySqlConnection();
    }

    // Faz alguma coisa
}
```

```
interface DBConnectionInterface {
    public function connect();
}

class MySqlConnection implements DBConnectionInterface {
    public function connect() { /*...*/ }
}

class OracleConnection implements DBConnectionInterface {
    public function connect() { /*...*/ }
}

class SQLServer implements DBConnectionInterface {
    public function connect() { /*...*/ }
}

class PasswordReminder {
    private $dbConnection;

    public function __construct( DBConnectionInterface $dbConnection ) {
        $this->dbConnection = $dbConnection;
    }

    // Faz alguma coisa
}
```

Problema 1:

- A refatoração que fizemos para o Princípio da Responsabilidade Única (SRP) e o Princípio Aberto/Fechado (OCP) já começou a pavimentar o caminho para o DIP.

```
public class FXMLDocumentController implements Initializable {  
    private final UsuarioService usuarioService = new UsuarioService(); // Depende da implementação concreta  
    private final UsuarioValidator usuarioValidator = new UsuarioValidator(); // Depende da implementação concreta  
    // ...  
}
```

- O Controller (módulo de alto nível, lida com a orquestração da UI) está dependendo diretamente das implementações concretas de UsuarioService e UsuarioValidator (módulos de baixo nível, lidam com lógica de negócios e validação).

Solução: Crie interfaces para UsuarioService e UsuarioValidator e faça com que o Controller dependa dessas interfaces. As instâncias concretas seriam injetadas (por construtor, setter, etc..)

```
// Interfaces (Abstrações)
public interface IUsuarioService {
    void cadastrarUsuario(UsuarioDTO usuario);
    void atualizarUsuario(UsuarioDTO usuario);
    void deletarUsuario(int id);
    ArrayList<UsuarioDTO> listarUsuarios();
}

public interface IUsuarioValidator {
    boolean validarUsuario(String nome, String email, String senha, String login);
    // Se houver métodos de validação individuais, também os coloque aqui
    boolean validarNome(String nome);
    boolean validarEmail(String email);
    boolean validarSenha(String senha);
    boolean validarLogin(String login);
}

// FXMLDocumentController (Módulo de Alto Nível) depende de Abstrações
public class FXMLDocumentController implements Initializable {
    private final IUsuarioService usuarioService; // Depende da interface
    private final IUsuarioValidator usuarioValidator; // Depende da interface

    // Injeção de dependência via construtor
    public FXMLDocumentController(IUsuarioService usuarioService, IUsuarioValidator usuarioValidator) {
        this.usuarioService = usuarioService;
        this.usuarioValidator = usuarioValidator;
    }
    // ...
}

// UsuarioService e UsuarioValidator (Módulos de Baixo Nível) implementam Abstrações
public class UsuarioService implements IUsuarioService {
    // ... implementação
}

public class UsuarioValidator implements IUsuarioValidator {
    // ... implementação
}
```

Benefício: Se você decidir trocar a implementação do UsuarioService (por exemplo, para um UsuarioServiceMock para testes, ou uma versão diferente que se conecta a uma API REST), o Controller não precisará ser modificado.

Como a Injeção de Dependência se encaixa com o carregamento de Views

```
// No ponto de entrada da sua aplicação (ex: método main ou classe principal do JavaFX)
public class MainApp {
    public static void main(String[] args) {
        // 1. Crie as implementações de baixo nível
        IUsuarioDAO usuarioDAO = new UsuarioDAO(); // Detalhe concreto

        // 2. Crie o serviço, injetando o DAO
        IUsuarioService usuarioService = new UsuarioService(usuarioDAO); // Abstração

        // 3. Crie o validador (já está implementando a interface)
        IUsuarioValidator usuarioValidator = new UsuarioValidator(); // Abstração

        // 4. Crie o controller, injetando o serviço e o validador
        // FXMLDocumentController controller = new FXMLDocumentController(usuarioService, usuarioValidator);
        // Em JavaFX, você precisaria de um Factory ou alguma forma de passar essas dependências
        // para o FXMLoader.
        // Exemplo simplificado para o FXMLoader:
        // FXMLoader loader = new FXMLoader(getClass().getResource("FXMLDocument.fxml"));
        // loader.setControllerFactory(c -> new FXMLDocumentController(usuarioService, usuarioValidator));
        // Parent root = loader.load();
    }
}
```


Exemplo usando fábrica de controladores

```
package application; // Ou onde sua classe principal do JavaFX está

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;
import controller.FXMLDocumentController;
import model.dao.IUsuarioDAO;
import model.dao.UsuarioDAO; // Importar a implementação concreta
import service.IUsuarioService;
import service.UsuarioService; // Importar a implementação concreta
import validator.IUsuarioValidator;
import validator.UsuarioValidator; // Importar a implementação concreta

import java.net.URL;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        // 1. Instancie as dependências concretas
        IUsuarioDAO usuarioDAO = new UsuarioDAO(); // Cria o DAO concreto
        IUsuarioService usuarioService = new UsuarioService(usuarioDAO); // Cria o serviço, injetando o DAO
        IUsuarioValidator usuarioValidator = new UsuarioValidator(); // Cria o validador concreto

        // 2. Configure o FXMLLoader com uma ControllerFactory
        FXMLLoader loader = new FXMLLoader();
        URL fxmlLocation = getClass().getResource("/view/FXMLDocument.fxml"); // Ajuste o caminho do seu FXML
        if (fxmlLocation == null) {
            System.err.println("Erro: FXMLDocument.fxml não encontrado. Verifique o caminho.");
            return;
        }
        loader.setLocation(fxmlLocation);

        loader.setControllerFactory(controllerClass -> {
            if (controllerClass == FXMLDocumentController.class) {
                // Retorna uma nova instância do seu controller, injetando as dependências
                return new FXMLDocumentController(usuarioService, usuarioValidator);
            }
            try {
                return controllerClass.newInstance();
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        });

        Parent root = loader.load(); // Carrega a view, usando a fábrica para criar o controller

        Scene scene = new Scene(root);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Cadastro de Usuários");
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Mais boas práticas:

- **DRY (Don't Repeat Yourself)**
- **KISS (Keep it Simple, Stupid)**
- **YAGNI (You Aren't Gonna Need It)**
- **Code Reviews**
- **Testes**
- **Comentários**
- **Documentação**

DRY

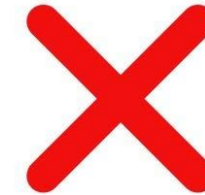
Fonte da imagem:
<https://medium.com/net-tips/dont-repeat-yourself-dry-principle-3c572f88733d>

Don't Repeat Yourself (DRY)

Principle

```
public class Car
{
    public int Year { get; set; }
    public string Model { get; set; } = string.Empty;
    public string Make { get; set; } = string.Empty;
    public int Tyre { get; set; }
}

public class Truck
{
    public int Year { get; set; }
    public string Model { get; set; } = string.Empty;
    public string Make { get; set; } = string.Empty;
    public int Tyre { get; set; }
}
```



mwaseemzakir



```
public class Vehicle
{
    public int Year { get; set; }
    public string Model { get; set; } = string.Empty;
    public string Make { get; set; } = string.Empty;
    public int Tyre { get; set; }
}

public class Car : Vehicle
{
    public int NumDoors { get; set; }
    public string EngineType { get; set; } = string.Empty;
}

public class Truck : Vehicle
{
    public bool HasFourWheelDrive { get; set; }
    public double PayloadCapacity { get; set; }
}
```

KISS – Keep it Simple, Stupid

KISS

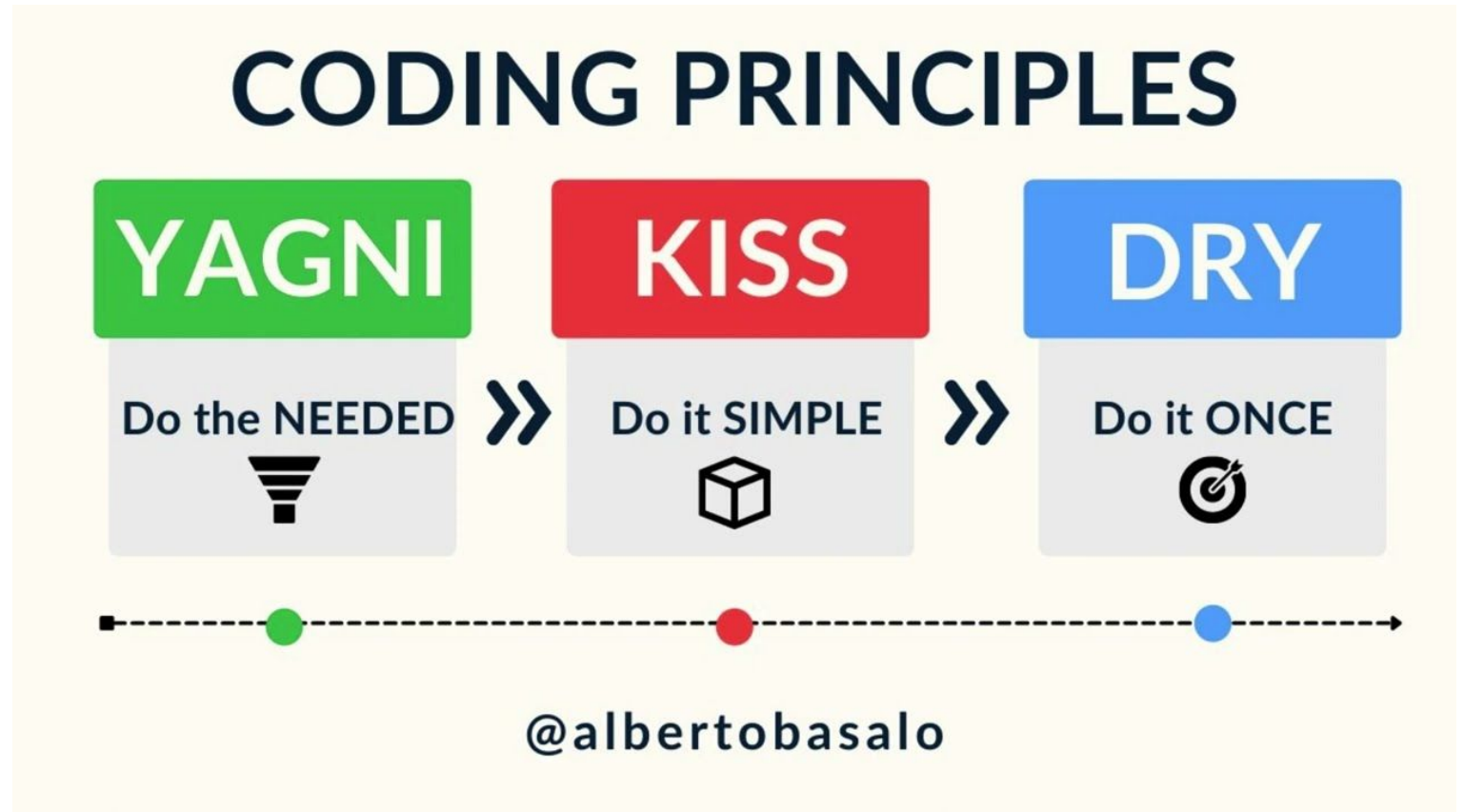
Fonte da imagem:
<https://www.interaction-design.org/literature/topics/keep-it-simple-stupid>



Interaction Design Foundation
[interaction-design.org](https://www.interaction-design.org)

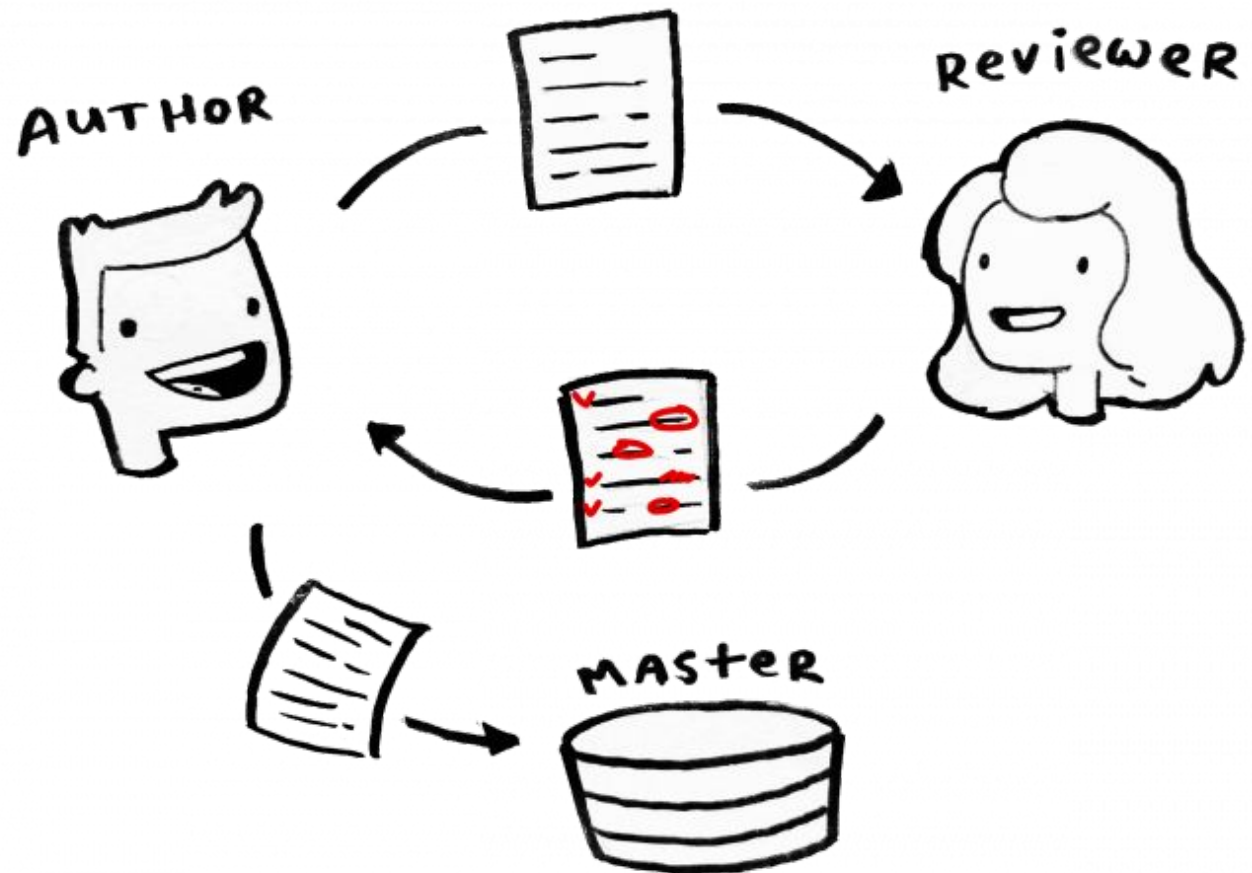
YAGNI

Fonte da imagem:
<https://www.interaction-design.org/literature/topics/keep-it-simple-stupid>



Code Reviews

Fonte da imagem:
<https://blog.innogames.com/recipes-for-better-code-reviews/>



Testes

- **Java:** JUnit;
- **Python:** PyTest;
- **C#:** NUnit.

Comentários e Documentação

- **Javadoc:** Java;
- **Doxygen:** C++, C, Java e Python;
- **Sphinx:** Python;

Design Patterns

- São soluções reutilizáveis para problemas comuns de design de software. São abordagens e estruturas que foram testadas e documentadas para resolver desafios específicos de programação. Esses padrões ajudam a melhorar a organização do código, a modularidade e a manutenibilidade do sistema.
- Assista:
https://www.youtube.com/watch?v=J-lHpiu-Twk&ab_channel=C%C3%B3digoFonteTV